# Agent Context Files: Field Guide

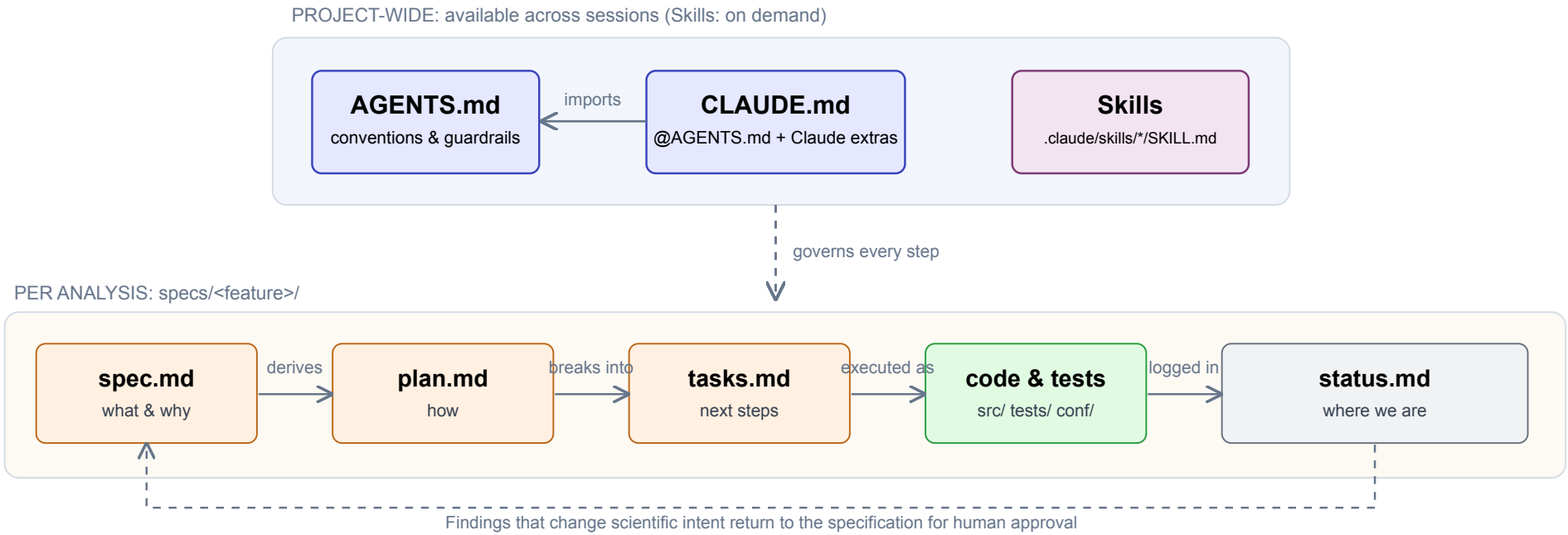
## At a glance

| File | Question it answers | What goes in it | When to create | Where to store | Who writes it; how often it changes |
|---|---|---|---|---|---|
| **AGENTS.md** | How do we work in this repo? | Project goals and scope, repository structure, tools and conventions, guardrails (quality checks, safety rules, domain constraints). Tool-neutral guidance shared across compatible agents. | Once, at project setup, after the repo and a short README exist. | Repo root. Optional nested instructions for subdirectories. | Human-owned; agent may draft. Changes when conventions or guardrails change. |
| **CLAUDE.md** | Same question, for Claude Code. | A thin file importing `AGENTS.md` with `@AGENTS.md`, plus Claude-specific instructions such as plan-mode expectations. | At setup: run `/init` in Claude Code, then review and customize. | Repo root (committed). Personal: `~/.claude/CLAUDE.md`. Local-only: `CLAUDE.local.md`. | Agent-generated draft, human-reviewed. Changes infrequently. |
| **spec.md** | What are we building, and why? | Scientific goal, scope, input versions, methodological decisions (CRS, boundary cells, weighting, statistic, NoData, day definition), rationale, outputs, constraints, acceptance criteria, open questions. Not implementation details. | Before implementation, after exploring the analysis. | `specs/<feature>/spec.md` | Agent may draft; human defines and approves decisions. Update when intent changes. |
| **plan.md** | How will we build it? | Technical approach, modules, data flow, libraries, configuration, data manifest, links from spec decisions to implementation, validation strategy, risks and alternatives. | After spec approval; before tasks. | `specs/<feature>/plan.md` | Agent drafts; human reviews. Update when design changes. |
| **tasks.md** | What is the next concrete step? | Ordered checklist of small tasks, each linked to a spec decision or acceptance criterion, with dependencies and a clear “done when” check. | After the plan is approved. | `specs/<feature>/tasks.md` | Agent updates task completion; human reviews and reprioritizes. |

| File | Question it answers | What goes in it | When to create | Where to store | Who writes it; how often it changes |
|---|---|---|---|---|---|
| **status.md** | Where did we leave off? | Current phase, completed and in-progress work, blockers, decisions awaiting approval, deviations, data status and next step. | When implementation begins; update at session end. | `specs/<feature>/status.md` | Agent updates at handoff; human reviews at session start. |

**Two kinds of files.** `AGENTS.md` and `CLAUDE.md` hold long-lived project conventions. `spec.md`, `plan.md`, `tasks.md`, and `status.md` capture decisions and progress for a specific analysis. **Skills** are reusable procedures invoked when relevant, not another analysis-specific decision record.

## How the files relate




| Read order for a new session | Write order for a new analysis | What goes where |
|---|---|---|
| 1. **AGENTS.md / CLAUDE.md:** project rules (loaded when supported/configured).<br>2. **status.md:** where the previous session stopped.<br>3. **tasks.md:** next unchecked task.<br>4. **spec.md / plan.md:** relevant decisions and implementation details. | 1. Explore the scientific problem.<br>2. Draft and refine **spec.md**; human approves.<br>3. Draft **plan.md**; human approves.<br>4. Create **tasks.md**.<br>5. Implement, validate, and update **status.md**.<br>6. Changes to intent go back to **spec.md**. | **AGENTS.md:** “Use type hints; plan first.”<br>**CLAUDE.md:** Claude-specific instructions.<br>**spec.md:** “Use area weighting.”<br>**plan.md:** “Use rasterio for zonal statistics.”<br>**tasks.md:** “Implement aggregate.py; done when coverage test passes.”<br>**status.md:** “Blocked: 3 counties have >5% NoData.”<br>**Skill:** reusable CRS or zonal-statistics procedure. |

**Specification spectrum.** *Ad hoc:* code first, no spec. *Spec-first:* a spec guides the initial build, then is abandoned. *Spec-anchored:* spec and code evolve together—the approach illustrated here. *Spec-as-source:* code is generated from the specification rather than hand-edited.

## What the templates look like

### AGENTS.md — project-wide rules

```
# Project
Reproducible county precipitation from
PRISM daily 800 m and 2020 Census blocks.

# Workflow
Hydra config + root Snakefile + Python modules.

# Scientific guardrails
Keep raw ZIPs immutable.
Validate CRS, grid, NoData, and date convention.
Use EPSG:5070 for overlap-area calculations.
PRISM day ends at 12:00 UTC.
```

### CLAUDE.md — Claude-specific rules

```
@AGENTS.md

# Feature workflow
Use create-specs.
Gate 1: approve spec.md before planning.
Gate 2: approve plan.md before coding.
Create tasks.md only after plan approval.
Reuse Skills; avoid scope creep.
Update status.md at handoff.
```

### spec.md — example: Feature 6

```
# Goal
Yearly county precipitation from Feature 5.

# Inputs
Validated daily county Parquet and Census blocks.

# Decisions
Yearly total = sum(daily means), mm.
Mean daily = total / days expected, mm/day.
Reject incomplete county-date coverage.

# Outputs and acceptance
One yearly Parquet + one county map.
Check dates and county IDs in both directions.
```

### plan.md — implementation approach

```
# Approach
Read Feature 5 daily Parquet.
Validate completeness before aggregation.
Group by county and PRISM year.
Dissolve Census blocks for map outlines.
Write yearly Parquet and PNG.

# Scope
Extend existing modules/QC/Snakefile.
Do not re-read PRISM rasters or
recompute Feature 4 crosswalk weights.
```

### tasks.md — short execution checklist

```
- [ ] Read daily county output
- [ ] Check expected dates and counties
- [ ] Calculate yearly total and mean
- [ ] Draw county choropleth
```

### status.md — session handoff

```
# Feature 6 status
Phase: implementation / verification / complete

Completed: [verified outputs]
```

```
- [ ] Run focused tests and QC
- [ ] Update status.md
```

```
Checks: [test results and QC]
Decisions: [approved changes only]
Blockers: [if any]
Next: [one concrete step]
```

**Keep each file distinct:** spec = scientific intent; plan = technical approach; tasks = execution checklist; status = what actually happened. These are concise examples, not replacements for the repository's approved specifications.

## Creating them in practice

| Step | What to do | Example in this pipeline |
|---|---|---|
| **1. Set up** | Create the repository, `AGENTS.md` and `CLAUDE.md`. Keep guidance short. | Document PRISM day definition, immutable raw ZIPs, CRS and Hydra/Snakemake conventions. |
| **2. Add Skills** | Keep reusable procedures under `.claude/skills/`; call them when relevant. | Use `create-specs`, raster-processing and spatial-aggregation Skills. |
| **3. Gate 1** | Ask the agent to inspect existing code and draft `spec.md`. Review scientific decisions; approve. | For Feature 4: grid IDs, block–cell overlap, area CRS, normalization and coverage. |
| **4. Gate 2** | Request `plan.md` with minimal modules, dependencies and tests. Approve before coding. | Reuse root `Snakefile`, shared `src/qc.py` and existing outputs. |
| **5. Implement** | Create `tasks.md`; work feature by feature. Do not rewrite earlier stages. | Download → Census → raster validation → crosswalk → daily → yearly. |
| **6. Validate** | Run focused tests, inspect QC and confirm an up-to-date Snakemake run plans no work. | For WA 2024: 366 days × 39 counties = 14,274 daily records. |
| **7. Hand off** | Update `status.md` with completed work, warnings and the next step. | A new agent session can resume without repeating the entire conversation. |

## Things worth knowing

- **Keep AGENTS.md / CLAUDE.md short.** They load into context repeatedly, so concise guidance saves tokens. Put detailed reusable instructions in Skills and feature-specific decisions in specs.
- **Do not duplicate.** Keep shared rules in AGENTS.md and reference them from CLAUDE.md. A symlink CLAUDE.md → AGENTS.md can work if no Claude-specific content is needed.
- **Use plan mode for the plan-first rule.** In Claude Code, Shift+Tab can switch to plan mode, where the agent proposes a plan without editing files. Use it alongside explicit spec and plan approval gates.
- **Methodological decisions belong in spec.md, not prompts.** Number each decision and record its rationale, such as area weighting, NoData handling and annual aggregation.

- **Acceptance criteria must be testable.** “Values look plausible” is not sufficient. Check measurable conditions, such as complete county–date coverage and defined raster-coverage thresholds; use the approved spec’s actual thresholds.
- **Skills vs spec.** A Skill stores reusable expertise (for example, how to validate raster grids); a spec records this feature’s decisions (for example, weighting choices). Write precise Skill descriptions so the agent knows when to load them.
- **Record data versions.** Recent PRISM dates may be revised for months. Record download date, stability status and checksums in manifests so reruns are explainable.
- **Commit everything.** Keep AGENTS.md, CLAUDE.md and each feature’s spec.md, plan.md, tasks.md and status.md in version control. For example, git log specs/04-prism-block-crosswalk/spec.md traces changes to that feature’s intent.
- **One spec folder per feature.** Use existing numbered folders, such as specs/04-prism-block-crosswalk/ and specs/05-county-daily-precip/; do not create a generic wa-precip folder or overwrite a previous feature’s spec.
- **status.md pairs with /clear and /compact.** Save completed work, decisions and next actions before reducing context, so a new session can resume without reconstructing the conversation.
- **Hierarchy in Claude Code.** ~/.claude/CLAUDE.md applies across projects; ./CLAUDE.md is repo-level; CLAUDE.local.md holds personal repo instructions; subdirectory CLAUDE.md files provide more local guidance. More specific instructions supplement general ones.
- **Repo-level TASKS.md vs feature tasks.md.** A root TASKS.md can track the overall project; specs/<feature>/tasks.md tracks the implementation of one feature. Keep both only if they serve distinct purposes.